

Inheritance

Advanced: Practice 1  

Which of the following statements best describes **inheritance** in object-oriented programming (OOP)?

- ☐ The process of encapsulating data and methods into a single unit, known as a class.
- ☐ The process of creating a new class from an existing class, that allows the new class to acquire its attributes and methods.
- ☐ A principle that emphasises the ability of objects to behave differently based on their data types.

Quiz:

STEM SMART Computer Science Week 22

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: sequence code

Advanced: Challenge 2  

The following class has been defined using pseudocode.

Pseudocode

```
1 CLASS Radio
2     PRIVATE volume: integer
3     PRIVATE station: string
4     PRIVATE on: Boolean
5
6     PUBLIC PROCEDURE Radio(given_station)
7         station = given_station
8         volume = 3
9         on = False
10    ENDPROCEDURE
11
12    PUBLIC FUNCTION get_volume()
13        RETURN volume
14    ENDFUNCTION
15
16    PUBLIC FUNCTION get_station()
17        RETURN station
18    ENDFUNCTION
19
20    PUBLIC FUNCTION is_on()
21        RETURN on
22    ENDFUNCTION
23
24    PUBLIC PROCEDURE set_volume(new_volume)
25        volume = new_volume
26    ENDPROCEDURE
27
28    PUBLIC PROCEDURE set_station(new_station)
29        station = new_station
30    ENDPROCEDURE
31
32    PUBLIC PROCEDURE switch()
33        IF on == True THEN
34            on = False
35        ELSE
36            on = True
37        ENDIF
38    ENDPROCEDURE
39
40 ENDClass
```

In testing, it was found that the volume of the radio could be set to an unsafe level. The `set_volume` method must be updated so that it does not allow the volume to exceed a setting of 30. Drag and drop the given statements to create an updated version of the method.

You must use **all of the statements** with **correct indentation** in your solution.

Available items

ENDIF

volume = 30

IF new_volume > 30 THEN

ELSE

ENDPROCEDURE

volume = new_volume

PUBLIC PROCEDURE set_volume(new_volume)

Quiz:
STEM SMART Computer Science Week 22

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: concepts 1

Advanced: Practice 2 ●●

OOP is a programming paradigm that differs from procedural programming. The statements that follow attempt to describe some core concepts of OOP but only **two** of the statements are accurate.

Select the **two accurate statements** from the list provided.

- ☐ A parent class inherits all of the attributes and methods of the child classes that it has.
- ☐ Access to the data of an object can be restricted using the **personal** and **public** access modifiers.
- ☐ To solve a problem, a program is divided into smaller parts called subroutines.
- ☐ A class is a series of step-by-step instructions that solve a problem.
- ☐ Multiple instances of a class can be created, each with different values for its attributes.
- ☐ The behaviour of a method that a child class has inherited from a parent class can be altered so that it behaves differently.

Quiz:

[STEM SMART Computer Science Week 22](#)

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: concepts 3

Advanced: Challenge 1



Alex wants to develop a simple video game where players can choose between different characters, each with unique abilities.

He decides to implement a superclass to represent common attributes shared by all characters, such as health points and movement speed, and their common behaviours. He then creates subclasses for each character type, such as "Warrior", "Mage", and "Archer" with their unique behaviours.

Which core OOP concept is Alex applying when he creates his subclasses?

- ☐ Encapsulation
- ☐ Polymorphism
- ☐ Abstraction
- ☐ Inheritance

Quiz:

STEM SMART Computer Science Week 22

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: concepts 2

Advanced: Challenge 2 

Marina is programming an application that aims to help students revise for their Biology lessons. The definitions she has written (so far) for the **Animal** and **Frog** classes of the program is presented below.

Pseudocode

```
1 CLASS Animal
2     PRIVATE habitat: String
3
4     PUBLIC FUNCTION get_habitat()
5         RETURN habitat
6     ENDFUNCTION
7
8     PUBLIC PROCEDURE display_habitat()
9         PRINT("My natural habitat is " + habitat)
10    ENDPROCEDURE
11 ENDCLASS
12
13 CLASS Frog EXTENDS Animal
14     PRIVATE secondary_habitat: String
15
16     PUBLIC FUNCTION get_secondary_habitat()
17         RETURN secondary_habitat
18     ENDFUNCTION
19
20     PUBLIC PROCEDURE display_habitat()
21         PRINT("I am an amphibian, I live in the " + habitat + " and also in the " + secondary_habitat)
22     ENDPROCEDURE
23 ENDCLASS
```

Select the OOP concepts that have been applied in this example.

- ☐ Encapsulation
- ☐ Polymorphism
- ☐ Decomposition
- ☐ Inheritance

Polymorphism

Advanced: Practice 2

Olivia maintains the computer systems for a car manufacturer who has traditionally made cars with internal combustion engines (ICE) but is branching out into the production of electric vehicles. She has used the technique of **polymorphism** in the design of her classes.

Which of the following examples uses polymorphism?

Example 1

Example 2

Example 3

Example 4

```
1 CLASS IceCar
2
3     PRIVATE tank_capacity
4     PRIVATE mpg
5     PRIVATE registration_number
6
7     PUBLIC PROCEDURE IceCar(given_reg_no, capacity, output)
8         registration_number = given_reg_no
9         battery_capacity = capacity
10        power_output = output
11    ENDPROCEDURE
12
13    PUBLIC FUNCTION get_registration()
14        RETURN registration_number
15    ENDFUNCTION
16
17    PUBLIC FUNCTION calculate_mileage()
18        RETURN tank_capacity / mpg
19    ENDFUNCTION
20 ENDCLASS
21
22 CLASS ElectricCar
23
24     PRIVATE battery_capacity
25     PRIVATE power_output
26     PRIVATE registration_number
27
28     PUBLIC PROCEDURE ElectricCar(given_reg_no, capacity, output)
29         registration_number = given_reg_no
30         battery_capacity = capacity
31         power_output = output
32    ENDPROCEDURE
33
34    PUBLIC FUNCTION get_registration()
35        RETURN registration_number
36    ENDFUNCTION
37
38    PUBLIC FUNCTION calculate_range()
39        RETURN battery_capacity * power_output
40
41
```

```
ENDFUNCTION
ENDCLASS
```

Example 1 Example 2 Example 3 Example 4

```
1 CLASS Car
2
3     PRIVATE registration_number
4
5     PUBLIC PROCEDURE Car(given_reg_no)
6         registration_number = given_reg_no
7     ENDPROCEDURE
8
9     PUBLIC FUNCTION get_registration()
10        RETURN registration_number
11    ENDFUNCTION
12
13    PUBLIC FUNCTION get_range()
14        RETURN NULL
15    ENDFUNCTION
16
17 ENDCLASS
18
19 CLASS IceCar EXTENDS Car
20
21     PRIVATE tank_capacity
22     PRIVATE mpg
23
24     PUBLIC PROCEDURE IceCar(given_reg_no, capacity, output)
25         SUPER(given_reg_no)
26         battery_capacity = capacity
27         power_output = output
28     ENDPROCEDURE
29
30     PUBLIC FUNCTION get_range()
31         RETURN tank_capacity / mpg
32     ENDFUNCTION
33 ENDCLASS
34
35 CLASS ElectricCar EXTENDS Car
36
37     PRIVATE battery_capacity
38     PRIVATE power_output
39
40     PUBLIC PROCEDURE ElectricCar(given_reg_no, capacity, output)
41         SUPER(given_reg_no)
42         battery_capacity = capacity
43         power_output = output
44     ENDPROCEDURE
45
46     PUBLIC FUNCTION get_range()
47         RETURN battery_capacity * power_output
48     ENDFUNCTION
49 ENDCLASS
```



```
1 CLASS Car
2
3     PRIVATE registration_number
4
5     PUBLIC PROCEDURE Car(given_reg_no)
6         registration_number = given_reg_no
7     ENDPROCEDURE
8
9     PUBLIC FUNCTION get_registration()
10         RETURN registration_number
11     ENDFUNCTION
12
13 ENDCLASS
14
15 CLASS IceCar EXTENDS Car
16
17     PRIVATE tank_capacity
18     PRIVATE mpg
19
20     PUBLIC PROCEDURE IceCar(given_reg_no, capacity, output)
21         SUPER(given_reg_no)
22         battery_capacity = capacity
23         power_output = output
24     ENDPROCEDURE
25
26     PUBLIC FUNCTION get_tank_capacity()
27         RETURN tank_capacity
28     ENDFUNCTION
29
30     PUBLIC FUNCTION get_mpg()
31         RETURN mpg
32     ENDFUNCTION
33
34 ENDCLASS
35
36 CLASS ElectricCar EXTENDS Car
37
38     PRIVATE battery_capacity
39     PRIVATE power_output
40
41     PUBLIC PROCEDURE ElectricCar(given_reg_no, capacity, output)
42         SUPER(given_reg_no)
43         battery_capacity = capacity
44         power_output = output
45     ENDPROCEDURE
46
47     PUBLIC FUNCTION get_battery_capacity()
48         RETURN battery_capacity
49     ENDFUNCTION
50
51     PUBLIC FUNCTION get_power_output()
52         RETURN power_output
53     ENDFUNCTION
54
55 ENDCLASS
```

[Example 1](#)[Example 2](#)[Example 3](#)[Example 4](#)

```
1 CLASS IceCar
2
3     PUBLIC tank_capacity
4     PUBLIC mpg
5     PUBLIC registration_number
6
7     PUBLIC PROCEDURE IceCar(given_reg_no, capacity, output)
8         registration_number = given_reg_no
9         battery_capacity = capacity
10        power_output = output
11    ENDPROCEDURE
12
13 ENDCLASS
14
15 CLASS ElectricCar
16
17     PUBLIC battery_capacity
18     PUBLIC power_output
19     PUBLIC registration_number
20
21     PUBLIC PROCEDURE ElectricCar(given_reg_no, capacity, output)
22         registration_number = given_reg_no
23         battery_capacity = capacity
24         power_output = output
25    ENDPROCEDURE
26
27 ENDCLASS
```

- ☐ Example 1
- ☐ Example 2
- ☐ Example 3
- ☐ Example 4

Quiz:

STEM SMART Computer Science Week 22

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Relationship between classes

Advanced: Challenge 2

Sam is creating a game where each player can choose the character (or sprite) that they can play with. A part of the definitions of the **Sprite** and **Game** classes is presented below. In the main program, an instance of the **Game** class called **my_game** is created.

Select the statement that correctly describes the type of relationship between the **Sprite** and **Game** classes.

Pseudocode

```
1 CLASS Sprite
2     PRIVATE score: Integer
3     PRIVATE name: String
4
5     PUBLIC PROCEDURE Sprite(given_name)
6         score = 0
7         name = given_name
8     ENDPROCEDURE
9
10    PUBLIC FUNCTION get_name()
11        RETURN name
12    ENDFUNCTION
13
14    PUBLIC FUNCTION get_score()
15        RETURN score
16    ENDFUNCTION
17 ENDClass
18
19 CLASS Game
20     PRIVATE my_sprite: Sprite
21
22     PUBLIC PROCEDURE Game()
23         my_sprite = NEW Sprite("Nikita")
24         PRINT(my_sprite.get_name())
25     ENDPROCEDURE
26 ENDClass
27
28 // Main program
29 PROCEDURE new_game()
30     my_game = NEW Game()
31 ENDPROCEDURE
```

- ☐ **Inheritance**, because through the **my_sprite** object, the **Game** class inherits all of the attributes and methods of the **Sprite** class.
- ☐ **Encapsulation**, because the **my_sprite** object is created within the **Game** class.

- ☐ **Composition**, because if the `my_game` object is destroyed, then the `my_sprite` object will also be destroyed.
 - ☐ **Aggregation**, because the `Game` class 'has a' `Sprite` object called `my_sprite`.
-
-
-

Quiz:

STEM SMART Computer Science Week 22

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: class diagram

Advanced: Challenge 1

Ben is writing an OOP program for an online chess game. He has sketched a **class diagram** to show the relationships between some of his classes. This diagram is shown in in **Figure 1**.

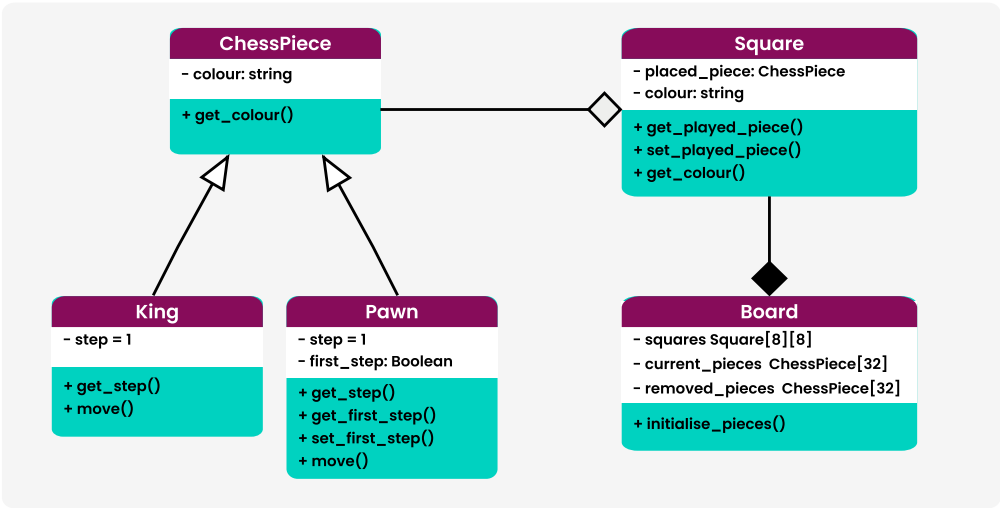


Figure 1: Ben's class diagram

The class diagram uses standard notation (UML) to show the relationship between the classes. These relationships are core OOP concepts.

Match each OOP concept to its description in the table.

OOP concept	Description
	Square objects are instantiated within the Board class and cannot exist separately from it.
	The implementation of the move method can be different for the King and Pawn classes, even though they have the same parent class.
	A King is a ChessPiece .
	A Square 'has a' ChessPiece , but the ChessPiece will already exist before it is placed on a Square , and it will cease to be linked to a specific Square as soon as it moves to a new one.

Items:

- Composition
- Aggregation
- Inheritance
- Polymorphism

OOP: benefits and drawbacks 1

Advanced: Challenge 1

All programming paradigms and languages have strengths, and also some weaknesses, and you need to understand these to pick the best option to work with.

Read each of the following statements and decide whether it is a correct statement relating to using an object-oriented programming language. Label each statement as **True** or **False** by dragging the correct label into the adjacent cell.

Statement	Label
1. OOP design techniques often make it easier to fully model a complete system.	
2. A system that relies on high volumes of message passing can degrade performance.	
3. Classes are modular, making maintenance easier.	
4. Encapsulation prevents direct access to private attributes.	
5. Classes cannot be extended to add extra functionality.	

Items:

True

False

Quiz:
STEM SMART Computer Science Week 22

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

